HDI · SF Bay Area · Vendor-health tooling

Pulse

Production-grade vendor-health monitoring for IT ops.

Thirty SaaS vendors. Sixty-second cadence. Two hundred seventy-six tests.
Built at Box IT as a Platform Engineer transition proof point.

Saagar Patel · Box IT

github.com/saagpatel/ITServiceHealth · MIT

01 · PROBLEM

The vendor-health signal is broken at the operator's desk.

30+ SaaS vendors

Identity, productivity, collaboration, engineering, HR, finance, sales, marketing, support. Every outage reaches the help desk before the status page does.

Status pages lag reality

Vendors mark themselves green while users flood #it-help. RSS is stale. JSON APIs don't agree on a schema. Manual updates go stale within the hour.

Slack becomes the war room

No dedup. No severity routing. Alerts arrive on the same channel as brownouts, flapping pollers, and vendor maintenance windows no one acknowledged.

Leadership asks: is it us?

Exec walks up to the big screen and wants one number. The NOC wall shows 80 service tiles and a timeline. That is not the meter a director reads.

One dashboard. Two views. Production-graded.

Pulse polls every vendor, normalizes five states, detects changes, writes audit events, and posts one clean Slack alert per real incident. Two views off the same data: Executive for the conference room, Engineer for the triage desk.

v1

Poll loop · 5-state normalizer · Slack Block Kit · React UI · dep graph · timeline · SLA

v2 · phase 0–1

Bearer-token admin auth · stamina retries · purgatory circuit breakers · poller_health · unknown-on-blind

v2 · phase 2

Flap suppression · dedup window · tier routing · dependency correlation · maintenance windows

v2 · phase 3

structlog JSON · Prometheus /metrics · Sentry · Healthchecks.io dead-man's switch

v2 · phase 4

aiosqlite pool · Litestream streaming · daily VACUUM INTO · retention · WAL checkpointing

v2 · phase 5–6

TanStack-style polling · Executive/Engineer toggle · PWA · a11y · CI · Caddy · Keychain

30
SaaS vendors

60 s
poll cadence

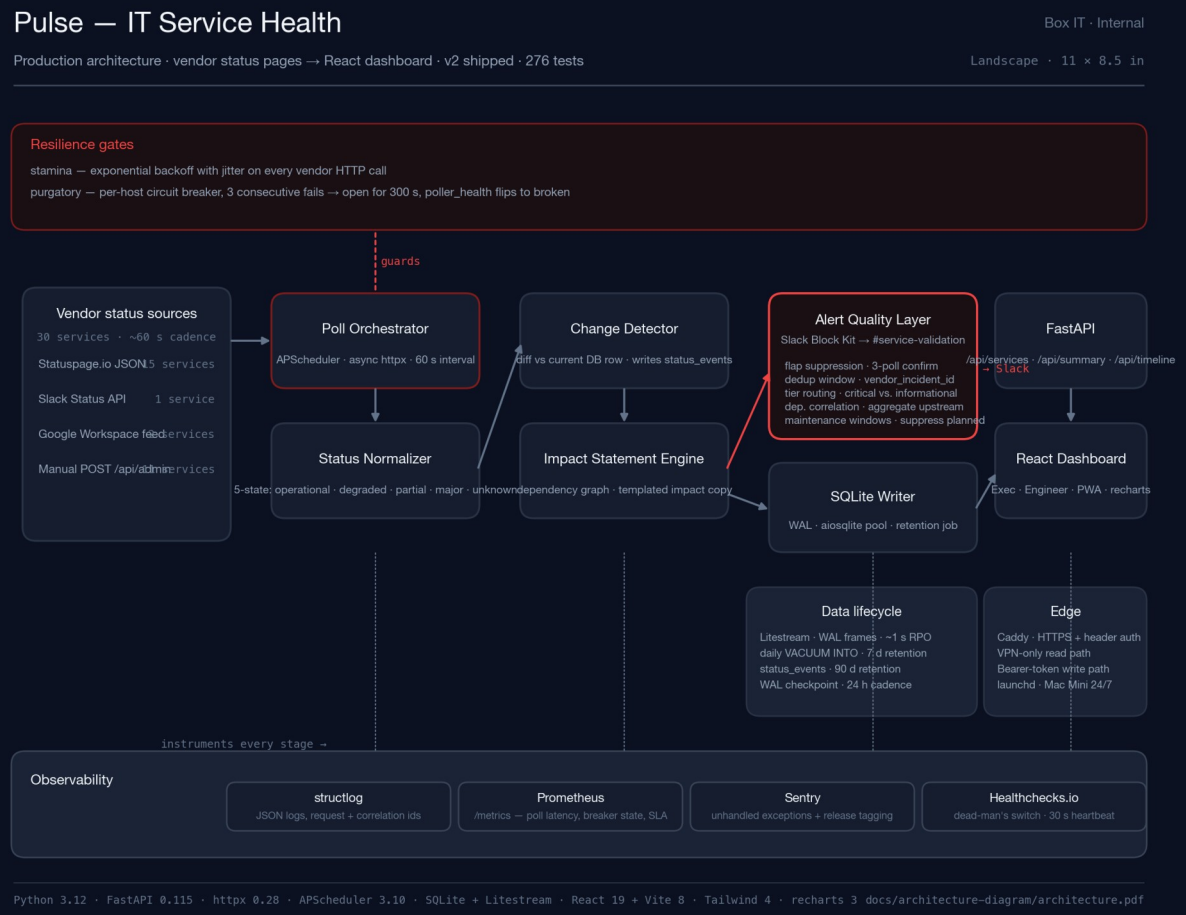
276
tests passing

~1 s
RPO · Litestream

One panel. Three meters. Read it from the back of the room.

Executive view · rendered at 1920x1080 · 2 active incidents

Vendor pages → resilient poll → alert hygiene → dashboard.



Landscape architecture · resilience + alert + observability lanes

Vendor APIs misbehave. The poller does not.

stamina

Exponential backoff with jitter on every outbound call.

· retry budget · 3 attempts with exponential base 0.5 s

· jitter · ±250 ms so we don't synchronise across services

· timeout · 10 s read · 5 s connect · no blocking I/O

· observability · every retry labeled with vendor + attempt

purgatory

Per-host circuit breaker. Blind is not operational.

· 3 consecutive failures · breaker opens for 300 s

· half-open probe · one attempt after TTL before closing

· poller_health flips · service renders as unknown, not operational

· Slack notification · separate channel from incident alerts

Rule: if we cannot see the vendor, we do not claim they are up. This is the single dashboard bug you cannot ship with.

One message per real incident. No one wakes up twice.

01	**Flap suppression**	3 consecutive confirming polls before worsening alerts fire. Minimum state duration of 600 s. Recovery requires 2 consecutive operational polls.
02	**Deduplication**	Alert dedup keyed on vendor_incident_id, not message text. One-day window by default. Same incident cannot re-page on every poll.
03	**Tier routing**	Critical vs informational. Tier picks destination channel and Slack priority. Degraded-brownout and major-outage do not share a notification surface.
04	**Dependency correlation**	When Okta goes down, we send one aggregated upstream alert instead of cascading alerts for every dependent service. Threshold is configurable.
05	**Maintenance windows**	First-class DB table. Scheduled vendor windows suppress alerts for the duration. Auto-populated from vendor feeds; manual windows supported.

Env-tunable: ALERT_CONFIRM_THRESHOLD_POLLS · ALERT_DEDUP_WINDOW_SECONDS · DEPENDENCY_CORRELATION_THRESHOLD

If the dashboard were down, I would know in 30 seconds.

structlog

`JSON logs`

Every request, poll, and alert carries a correlation id. WatchedFileHandler survives logrotate.

Prometheus

`/metrics exposition`

Poll latency, breaker state, alert dispatches, SLA percentage per service. Scrapeable by any text-format collector.

Sentry

`Exception capture`

Unhandled exceptions reported with release tag. Traces optional. Default sample rate zero.

Healthchecks.io

`Dead-man's switch`

30-second heartbeat. /healthz returns 503 past 120 s. External observer notices silence even if monitoring itself is dead.

Rule: every layer reports independently. The poller, the API, the writer, and the browser all answer the question is this alive?

Shipped. In tree. Production-graded. Boring by design.

7 production phases shipped

276 tests passing

~1 s RPO · Litestream streaming

< 30 s p50 vendor-status detection

Before → After

Detection	user reports in Slack	→	**60 s poll + dedup alert**
Reliability	vendor page = truth	→	**blind = unknown, not green**
Exec read	80-tile NOC grid	→	**one panel · three meters**
Alerts	one per poll	→	**one per real incident**
Backups	none	→	**~1 s RPO + daily snapshot**

Not shipped: LLM summarization, Splunk / JSM / ThousandEyes / Datadog. Deferred on purpose.

09 · PLATFORM ENGINEER LENS

This is what the job looks like from this side of the help-desk.

Pulse is not a side project. It is the shape of the work: listen to vendor signal end-to-end, design resilience gates before you write business logic, refuse to claim green when you are blind, put one meter in the room that a director can read. IT support told me where the pain was. Platform engineering is what I did with it.

If you operate

Steal the five alert-hygiene layers.

If you procure

Ask vendors for these five guarantees, not feature lists.

If you build

Clone it. MIT. github.com/saagpatel/ITServiceHealth.